

GHANA COMMUNICATION TECHNOLOGY UNIVERSITY (GCTU)

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

**DESIGN AND EVALUATION OF A SIMULATION-
BASED ADAPTIVE SMART TRAFFIC-LIGHT
CONTROLLER**

A Consolidated Final-Year Project Report

SUBMITTED BY

[STUDENT NAME] - ID [STUDENT ID]

SUPERVISOR

[SUPERVISOR NAME]

SEPTEMBER 2026

Declaration

This report describes a simulation-based adaptive traffic-light control project developed for academic evaluation. The implementation is not intended for use on public roads or as certified traffic-control equipment.

Abstract

This project designs, implements, and evaluates an adaptive smart traffic-light controller for a four-way intersection using a full software simulation. The system compares an adaptive demand-based controller with a fixed-time baseline under repeatable traffic scenarios. The simulator records arrivals, completed vehicles, waiting time, maximum queue length, detector-fault

behaviour, and safety violations. A browser dashboard provides live visualisation of signal phases, vehicle queues, detector values, benchmark tables, and aggregate charts.

The final benchmark used five traffic scenarios, 10 random seeds, 300-second runs, and a 0.5-second simulation step. Adaptive control reduced mean waiting time in every tested scenario, with improvements from 8.33% to 43.11%. It also completed more vehicles than the fixed-time baseline in every scenario. The controller recorded zero conflicting-green violations across the benchmark and detector-fault experiments.

Table of Contents

1. CHAPTER ONE: INTRODUCTION
2. CHAPTER TWO: LITERATURE REVIEW
3. CHAPTER THREE: SYSTEM SPECIFICATION AND DESIGN
4. CHAPTER FOUR: SYSTEM IMPLEMENTATION, PRESENTATION EVIDENCE, AND TESTING
5. CHAPTER FIVE: CONCLUSION, RECOMMENDATIONS, AND FUTURE WORK
6. REFERENCES
7. APPENDICES

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Urban intersections are shared spaces where multiple traffic streams compete for right of way. Traffic-light controllers must allocate movement time efficiently while preventing unsafe

conflicts. Conventional fixed-time traffic lights follow predetermined timings, which makes them

simple and predictable. However, fixed timings cannot react when traffic demand changes from one

approach to another.

An adaptive traffic-light controller can respond to measured traffic demand by adjusting green duration. For example, if North/South queues are much longer than East/West queues, the controller

can allocate more green time to North/South traffic while preserving minimum green, maximum green,

amber, and all-red clearance rules.

1.2 Problem Statement

Fixed-time signal control can waste green time when demand is uneven or changing. A fixed-time

phase may continue serving a low-demand approach while longer queues build on another approach.

The engineering problem is to design a controller that responds to demand without violating signal-safety rules.

The previous hardware direction also created a project-risk problem. Physical sensors, Raspberry Pi wiring, and LED demonstration hardware would show a working prototype, but they would not by

themselves prove whether the adaptive controller performs better than a fixed-time baseline. The approved simulation-only direction allows the project to focus on measurable algorithm behaviour.

1.3 Aim of the Study

The aim is to build and evaluate an adaptive four-way smart traffic-light controller using repeatable simulation, live visualisation, database logging, and fixed-vs-adaptive benchmarks.

1.4 Objectives of the Study

8. Implement a safe traffic-light phase controller for a four-way intersection.
9. Implement a fixed-time baseline controller.
10. Implement an adaptive controller that changes green duration based on detected demand.
11. Build a repeatable traffic simulator with multiple demand scenarios.

12. Log simulation runs, phase events, detector samples, and traffic metrics.
13. Provide a browser dashboard showing signals, queues, detector values, and benchmark charts.
14. Compare fixed-time and adaptive controllers using identical traffic traces.
15. Evaluate detector-fault behaviour and signal safety.

1.5 Justification of the Study

This project is engineering-related because it designs, builds, tests, and evaluates a complete software system. It does not claim that adaptive traffic-light control is new. The contribution is the implemented system and its evidence: controller logic, safety constraints, simulation, fault injection, persistence, API, dashboard, benchmark scripts, generated results, and automated tests. Compared with a basic traffic-light demonstration, this project adds fair fixed/adaptive comparison, repeatable scenarios, detector-fault testing, true-queue versus detector-demand separation, and result artifacts that can be regenerated.

1.6 Significance of the Study

The project gives a controlled way to demonstrate how adaptive green-time allocation can improve

traffic performance under uneven and changing demand. It also shows that adaptive timing should be

separated from signal safety: the adaptive algorithm may influence green duration, but the shared phase controller still enforces amber and all-red clearance.

1.7 Scope of the Study

The system models a single four-way intersection with North, East, South, and West approaches. North/South and East/West are treated as opposing movement groups. The controller includes green,

amber, and all-red clearance states.

The current scope excludes Raspberry Pi deployment, GPIO wiring, physical LEDs, camera hardware,

pedestrian phases, emergency-vehicle priority, and public-road operation. The project is an academic simulation and evaluation system.

1.8 Limitations of the Study

The simulator is simplified. It considers rain, darkness, glare, dirt, and similar environmental

conditions as possible causes of incorrect detector readings, but it does not physically model image quality, sensor hardware, road-surface conditions, or visibility. It also does not model turning movements, pedestrians, emergency vehicles, lane changes, vehicle classes, realistic driver behaviour, calibrated road geometry, or detailed detector noise.

1.9 Methodology Overview

The project was implemented incrementally. The safe phase controller was built first, followed by

the fixed-time baseline, adaptive timing policy, simulator, storage, API, dashboard, benchmarks, fault experiments, and report export workflow. The implementation was then evaluated using tests,

dashboard screenshots, CSV outputs, and benchmark charts.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

Traffic-signal timing affects delay, queue length, throughput, and safety at intersections. This chapter summarises the concepts that support the project design: fixed-time control, adaptive signal control, phase safety, simulation-based evaluation, and the software tools used to implement the system.

2.2 Fixed-Time Signal Control

Fixed-time traffic-light control uses predetermined timings for each phase. This provides a clear baseline because the controller does not depend on live demand. In this project, the fixed-time baseline uses the same safety phase sequence as the adaptive controller, but every green phase uses the configured fixed duration.

2.3 Adaptive Signal Control

Adaptive signal control changes timing in response to traffic conditions. The Federal Highway Administration describes adaptive signal control technology as a way to adjust signal timing to changing traffic patterns.[fhwa-asct] This project follows the same broad concept but applies it inside a lightweight academic simulator.

The adaptive controller uses detected queue demand to choose a target green time between minimum

and maximum limits. It is intentionally transparent rather than machine-learning based, making the decision process easier to inspect and defend.

2.4 Signal Timing and Safety

Signal timing includes more than green allocation. FHWA traffic-signal timing guidance discusses

minimum green, maximum green, yellow-change, and red-clearance intervals as separate timing elements.[fhwa-timing] This supports the project's explicit phase model:

all-red, green, amber, all-red, opposing green

The project separates adaptive timing from signal safety. Adaptive logic decides when a green phase should end, but it cannot skip amber or all-red clearance.

2.5 Simulation-Based Evaluation

Simulation allows controlled comparison between controllers. The same random seed can be used for

both fixed-time and adaptive runs, ensuring both controllers receive the same arrival trace. This is important because otherwise a controller could appear better simply because it received easier traffic input.

Traffic simulation is also widely used in transport research. SUMO provides traffic-light modelling and supports simulation of signal control algorithms.[sumo-tl] This project uses a custom simulator because it is smaller, easier to test, and closely aligned with the project requirements.

2.6 Software Tools

FastAPI is used for HTTP and WebSocket interfaces.[fastapi-docs][fastapi-ws] SQLite is used as an embedded database for local run logging.[sqlite-wal] pytest is used for automated testing and supports parametrised test cases for repeated checks across scenarios.[pytest-parametrize]

2.7 Research Gap and Project Difference

Many existing traffic-light projects demonstrate signal switching or describe adaptive timing.

The gap addressed here is a compact, repeatable, simulation-only implementation that connects the

controller, scenario generator, detector fault model, dashboard, persistence layer, benchmark

runner, generated charts, and automated tests into one defensible final-year project.

CHAPTER THREE: SYSTEM SPECIFICATION AND DESIGN

3.1 System Boundary

The system includes the controller logic, traffic simulator, fault injection, SQLite logging, FastAPI API, browser dashboard, benchmark runner, and report export scripts. It does not include

real traffic-light hardware.

3.2 System Architecture

Layer	Purpose	Main files
Domain	Signal phases, controller timing, safety rules	src/trafficlight/domain/
Simulation	Queue model, traffic scenarios, faults, benchmarks	src/trafficlight/simulation/
Storage	SQLite run/event/metric logging	src/trafficlight/storage/
Interface	CLI, FastAPI endpoints, WebSocket stream, dashboard	src/trafficlight/main.py, src/trafficlight/api/ src/trafficlight/dashboard/

3.3 Functional Requirements

ID	Requirement
FR-01	The system shall model a four-way intersection with North, East, South, and West approaches.
FR-02	The controller shall support North/South and East/West movement groups.
FR-03	The controller shall start in an all-red state.
FR-04	The controller shall transition through green, amber, and all-red clearance states.
FR-05	The system shall provide a fixed-time baseline controller.
FR-06	The system shall provide an adaptive demand-based controller.
FR-07	The adaptive controller shall enforce minimum and maximum green durations.

FR-08	The simulator shall generate repeatable arrivals using scenario configuration and seeds.
FR-09	The simulator shall track arrivals, departures, waiting time, queue length, and throughput.
FR-10	The system shall include balanced, asymmetric, burst, and changing-demand scenarios.
FR-11	The system shall support stuck-high and stuck-low detector fault profiles.
FR-12	The dashboard shall show live signals, queues, vehicles, detector values, and benchmark results.
FR-13	The benchmark runner shall export row-level and aggregate CSV results.
FR-14	The reporting workflow shall generate SVG charts and markdown summaries.

3.4 Safety Requirements

ID	Requirement
SR-01	North/South green and East/West green shall never be active at the same time.
SR-02	Every applied signal state shall pass through the safety check.
SR-03	Adaptive timing shall not bypass amber or all-red clearance states.
SR-04	Detector faults shall not directly command signal colours.
SR-05	The safety target for benchmark and fault experiments shall be zero conflicting-green violations.

3.5 Detector and Environmental Assumptions

Because the approved project scope is simulation-only, vehicles are detected through a virtual detector model rather than through cameras, ultrasonic sensors, inductive loops, or GPIO input.

The simulator maintains the true queue length on each approach, then exposes a controller-visible

demand value. The controller uses this demand value to choose green duration.

Factor	Treatment in current simulation
--------	---------------------------------

Heavy traffic on one axis	Modelled through ns-heavy and ew-heavy demand scenarios
Short demand surge	Modelled through the ns-burst scenario
Peak direction changing over time	Modelled through the alternating-peak scenario
Sensor stuck high	Modelled through the ns-stuck-high fault profile
Sensor stuck low or missed detections	Modelled through ew-stuck-low and all-stuck-low-midrun
Rain, darkness, glare, dirt, or poor visibility	Considered as possible causes of incorrect detector demand, but not physically modelled
Pedestrians, emergency vehicles, turning lanes, blocked roads, and lane changes	Outside the current scope and listed as future work

3.6 Interface Requirements

The project can be used through the command line, HTTP API, WebSocket stream, and browser dashboard. The dashboard is the main demonstration interface because it shows signal colours, phase state, vehicles, queue values, detector readings, run history, and benchmark charts.

CHAPTER FOUR: SYSTEM IMPLEMENTATION, PRESENTATION EVIDENCE, AND TESTING

4.1 Implementation Environment

The implementation uses Python for the controller and simulator, FastAPI for the API, SQLite for

local persistence, pytest for automated testing, and static HTML/CSS/JavaScript for the dashboard.

The project runs on a normal laptop without physical traffic-light hardware.

4.2 Controller Implementation

Both controllers share the same base phase sequence:

16. all-red before North/South green
17. North/South green
18. North/South amber
19. all-red before East/West green
20. East/West green
21. East/West amber

The fixed-time controller uses the configured fixed green duration. The adaptive controller

calculates target green duration using the active movement group's share of total demand. The adaptive value is clamped between minimum and maximum green limits.

Parameter	Value
Minimum green	8 seconds
Fixed green baseline	20 seconds
Maximum green	45 seconds
Amber	3 seconds
All-red clearance	2 seconds

4.3 Simulation Engine

The simulator advances in discrete time steps. At each step, it adds arrivals, applies detector faults if configured, advances the controller, checks safety, discharges vehicles on green approaches, accumulates waiting time, logs metrics, and sends dashboard updates.

Scenario	Description
balanced	Similar demand on all four approaches
ns-heavy	North/South demand is much higher than East/West
ew-heavy	East/West demand is much higher than North/South
ns-burst	A short North/South surge during moderate demand
alternating-peak	Demand shifts from North/South to East/West during the run

4.4 Dashboard and Presentation Evidence

The dashboard is a static HTML/CSS/JavaScript interface served by FastAPI. It uses WebSocket messages for live simulation updates and HTTP requests for benchmark execution. It visualises signal lamps, queue meters, detector values, visible vehicles, run history, benchmark tables, and aggregate charts.

Presentation evidence includes:

- output/playwright/dashboard-live-simulation.png
- output/playwright/dashboard-fault-mismatch.png
- output/playwright/dashboard-benchmark-results.png
- docs/presentation/traffic-light-demo-corrected.pptx

4.5 Testing and Validation

The automated test suite covers:

- controller startup and phase transitions;
- conflicting-green rejection;
- adaptive maximum-green enforcement;
- adaptive demand response;
- simulation comparability across identical seeds;
- SQLite logging;
- API and WebSocket behaviour;
- benchmark aggregation and report generation;
- detector fault behaviour.

Latest result:

24 passed, 1 warning

The warning is a Starlette TestClient deprecation warning and does not indicate a project test failure.

4.6 Results and Evaluation

The final benchmark used 300-second runs, 0.5-second simulation steps, and seeds 1 to 10. Each fixed/adaptive comparison used the same scenario and seed.

The final experiment package was regenerated after correcting the simulator's arrival and waiting time accounting. Arrivals are now sampled from the configured per-minute demand rate, and mean

wait is calculated from completed vehicles using each vehicle's recorded arrival and departure time.

Scenario	Fixed mean wait (s)	Adaptive mean wait (s)	Wait improvement	Completed change	Safety violations
Balanced	23.91 +/- 1.25	21.89 +/- 1.25	+8.33% +/- 3.90 pp	+1.90 +/- 1.90	0
NS-heavy	43.93 +/- 3.38	29.51 +/- 2.37	+32.54% +/- 4.23 pp	+41.60 +/- 3.67	0
EW-heavy	50.15 +/- 2.77	28.48 +/- 2.15	+43.11% +/- 3.57 pp	+42.70 +/- 3.33	0
NS-burst	44.10 +/- 1.71	37.82 +/- 3.21	+14.46% +/- 5.07 pp	+36.50 +/- 3.18	0
Alternating peak	54.58 +/- 3.50	38.09 +/- 5.24	+30.76% +/- 6.20 pp	+23.50 +/- 2.98	0

Adaptive control reduced mean waiting time in all five scenarios and completed more vehicles in all five scenarios.

Detector-fault experiments used the NS-heavy scenario with the same duration, step size, and seed set.

Fault profile	Runs	Mean completed	Mean wait (s)	Mean max queue	Safety violations
none	10	196.50	29.51	22.90	0
ns-stuck-high	10	198.40	26.52	17.40	0
ew-stuck-low	10	197.50	24.29	17.30	0
all-stuck-low-midrun	10	160.60	34.18	41.00	0

The all-stuck-low-midrun fault produced the worst performance degradation. However, all fault experiments still recorded zero conflicting-green violations.

CHAPTER FIVE: CONCLUSION, RECOMMENDATIONS, AND FUTURE WORK

5.1 Summary of the Project

The project successfully delivered a simulation-only adaptive smart traffic-light controller. It includes a safe phase state machine, fixed-time baseline, adaptive demand-based timing, repeatable scenarios, detector fault simulation, SQLite logging, FastAPI API, WebSocket dashboard, benchmark runner, report exports, and automated tests.

5.2 Evaluation of Project Objectives

All main project objectives were implemented for the approved simulation-only scope. The system models the four-way intersection, compares fixed and adaptive control, logs results, presents a browser dashboard, exports report artifacts, and tests detector faults.

5.3 Major Achievements

- The adaptive controller reduced mean waiting time in every benchmark scenario.
- The adaptive controller completed more vehicles than the fixed-time baseline in every scenario.
- The simulation recorded zero conflicting-green violations across benchmark and detector-fault experiments.
- The dashboard showed both true queue values and controller-visible detector values.

- The final result package can be regenerated from command-line scripts.

5.4 Key Findings

The results support the central project claim. Adaptive control improved mean waiting time under

balanced, asymmetric, burst, and changing-demand conditions while preserving safety. The adaptive controller improved completed-vehicle wait, maximum queue, and throughput across the

benchmark set, while detector-fault experiments showed that poor detector input can still reduce performance.

Detector faults affected performance but did not create unsafe signal states. This is because faults change only the demand input seen by the controller. They do not bypass the phase controller or directly set lamp colours.

5.5 Limitations of the Project

The project uses a simplified traffic simulator. It considers weather, darkness, glare, and similar environmental factors as possible causes of detector error, but it does not physically model image quality, sensor hardware, road-surface conditions, or visibility. The results should therefore be interpreted as evidence for the implemented simulation rather than direct predictions for a real road junction.

5.6 Recommendations

- Keep the project framed as a simulation and evaluation system, not as public-road equipment.
- Use the fixed-time baseline when explaining why the adaptive controller is an improvement.
- Use detector-fault screenshots to explain how virtual detection replaces physical sensors.
- Present the zero-conflicting-green result as a safety property of the phase machine.
- Avoid claiming that the project invented adaptive signal control.

5.7 Future Work

Future work could add:

- pedestrian request phases;
- emergency-priority simulation;
- richer detector noise models;
- sensitivity tests for arrival rate and discharge headway;

- calibrated field or model-road data;
- SUMO-based validation;
- optional hardware adapters if the project later returns to Raspberry Pi demonstration.

5.8 Final Conclusion

The final experiments show that adaptive control reduced mean waiting time by 8.33% to 43.11%

across the tested scenarios. The adaptive controller also completed more vehicles than the fixed-time baseline in every scenario. Across all benchmark and detector-fault experiments, the system recorded zero conflicting-green violations. The project therefore meets the approved simulation-only objective and provides a defensible engineering contribution through a tested, repeatable, and evidence-based implementation.

REFERENCES

- [fhwa-asct]: Federal Highway Administration, "Adaptive Signal Control Technology",
<https://www.fhwa.dot.gov/innovation/everydaycounts/edc-1/asct.cfm>
- [fhwa-timing]: Federal Highway Administration, "Traffic Signal Timing Manual: Chapter 5",
<https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter5.htm>
- [sumo-tl]: Eclipse SUMO documentation, "Traffic Lights",
https://sumo.dlr.de/docs/Simulation/Traffic_Lights.html
- [fastapi-docs]: FastAPI documentation, "FastAPI",
<https://fastapi.tiangolo.com/>
- [fastapi-ws]: FastAPI documentation, "WebSockets",
<https://fastapi.tiangolo.com/advanced/websockets/>
- [sqlite-wal]: SQLite documentation, "Write-Ahead Logging",
<https://www.sqlite.org/wal.html>
- [pytest-parametrize]: pytest documentation, "Parametrizing tests",
<https://docs.pytest.org/en/stable/example/parametrize.html>

APPENDICES

Appendix A: Generated Result Artifacts

Artifact	Purpose
results/dissertation/results_summary.md	Markdown experiment summary
results/dissertation/benchmark_rows.csv	Row-level fixed/adaptive benchmark data
results/dissertation/benchmark_summary.csv	Aggregate benchmark statistics
results/dissertation/failure_modes.csv	Detector-fault experiment data
results/dissertation/mean_wait.svg	Mean waiting-time chart
results/dissertation/max_queue.svg	Maximum queue chart
results/dissertation/completed.svg	Completed vehicles chart

Appendix B: Dashboard Evidence

Artifact	Purpose
output/playwright/dashboard-live-simulation.png	Live simulation with visible vehicles
output/playwright/dashboard-fault-mismatch.png	Detector fault showing queue/demand mismatch
output/playwright/dashboard-benchmark-results.png	Benchmark results table and aggregate charts

Appendix C: Important Commands

Run tests:

```
.\.venv\Scripts\python.exe -m pytest
```

Run dashboard:

```
.\.venv\Scripts\python.exe -m uvicorn trafficlight.api.app:app --host 127.0.0.1 --port 8000
```

Generate dissertation results:

```
.\.venv\Scripts\python.exe scripts\dissertation_results.py --duration 300 --step 0.5 -  
-seeds 10 --output-dir results\dissertation
```